Kurs Django — Lekcja 7

Formularze (Django Forms)

Cel lekcji

Student rozumie różnicę między forms.Form a forms.ModelForm, potrafi zwalidować dane i obsłużyć formularz w widoku (GET/POST), rozumie ochronę CSRF.

Blok 1: Teoria (10–15 min)

forms.Form vs forms.ModelForm

- forms.Form — definiujemy pola ręcznie, niezależnie od modelu. Przydatne np. do formularza kontaktowego, który nie zapisuje się bezpośrednio do bazy.
- forms.ModelForm — generuje formularz automatycznie na podstawie modelu. Mniej kodu, mniej powtórzeń — idealne do CRUD-a na modelach, które już mamy.

Cykl GET/POST

- GET — użytkownik wchodzi na stronę, dostaje pusty (lub wstępnie wypełniony) formularz
- POST — użytkownik wysyła dane, widok je waliduje i zapisuje (albo zwraca formularz z błędami)

CSRF (Cross-Site Request Forgery)

Django domyślnie chroni formularze przed atakami, w których złośliwa strona próbuje wysłać żądanie w imieniu zalogowanego użytkownika. Dlatego w każdym formularzu HTML trzeba umieścić {% csrf_token %} — bez tego POST zostanie odrzucony.

Blok 2: Ćwiczenia w terminalu

Krok 1 — prosty forms.Form (formularz kontaktowy)

Nowy plik blog/forms.py:

```
from django import forms

class KontaktForm(forms.Form):
    imie = forms.CharField(max_length=100, label="Imię")
    email = forms.EmailField(label="Adres e-mail")
    wiadomosc = forms.CharField(widget=forms.Textarea, label="Wiadomość")
```

Krok 2 — widok obsługujący formularz

W blog/views.py:

```
from .forms import KontaktForm

def kontakt(request):
```

```
    if request.method == 'POST':
        form = KontaktForm(request.POST)
        if form.is_valid():
            imie = form.cleaned_data['imie']
            email = form.cleaned_data['email']
            wiadomosc = form.cleaned_data['wiadomosc']
            # na razie tylko wypisujemy w konsoli, wysyłka maila to osobny temat
            print(f"Nowa wiadomość od {imie} ({email}): {wiadomosc}")
            return render(request, 'blog/kontakt_sukces.html')
    else:
        form = KontaktForm()

    return render(request, 'blog/kontakt.html', {'form': form})
```

Krok 3 — szablon formularza

templates/blog/kontakt.html:

```
{% extends "base.html" %}
{% block content %}
    <h2>Skontaktuj się z nami</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Wyślij</button>
    </form>
{% endblock %}
```

templates/blog/kontakt_sukces.html:

```
{% extends "base.html" %}
{% block content %}
    <p>Dziękujemy! Wiadomość została wysłana.</p>
{% endblock %}
```

Krok 4 — dodanie URL-a

W blog/urls.py:

```
path('kontakt/', views.kontakt, name='kontakt'),
```

Krok 5 — test walidacji

```
python manage.py runserver
```

Wejść na /kontakt/, spróbować wysłać pusty formularz lub niepoprawny e-mail — Django samo wyświetli komunikaty błędów przy polach.

Krok 6 — ModelForm dla modelu Post

```
from .models import Post

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['tytul', 'tresc', 'kategoria', 'tagi', 'opublikowany']
```

Krok 7 — widok dodawania posta przez ModelForm

```
def dodaj_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('strona_glowna')
    else:
        form = PostForm()

    return render(request, 'blog/dodaj_post.html', {'form': form})
```

Pamiętać o imporcie redirect:

```
from django.shortcuts import render, redirect
```

URL:

```
path('dodaj/', views.dodaj_post, name='dodaj_post'),
```

Blok 3: Zadanie dla studentów

- Dodać do KontaktForm pole temat (wybór z listy przez forms.ChoiceField, np. „Pytanie”, „Reklamacja”, „Inne”)
- Dodać prostą walidację niestandardową w KontaktForm — metodę clean_wiadomosc, która odrzuca wiadomości krótsze niż 10 znaków
- Stworzyć formularz do dodawania Komentarz (z Lekcji 6) jako ModelForm i podpiąć go pod widok szczegółów posta (na razie wystarczy osobny URL /post/<id>/komentarz/)

Checklist nawyków

☐ Każdy formularz HTML z metodą POST ma {% csrf_token %}

☐ Widok obsługujący formularz rozróżnia GET i POST

☐ Po form.is_valid() dane brać z form.cleaned_data, a nie bezpośrednio z request.POST